🪞 Autoevaluación

Tabla de contenidos

1 Funciones 1
1.1 Puras 1
1.2 Anónimas 1
1.3 Variádicas 2
1.4 *Closures* 3
2 Recursión 3
3 Funciones de orden superior 3
4 Comprehensions 5
5 Generadores 6

1 Funciones

1.1 Puras

1. ¿Por qué decimos que `fun` es una función con efectos secundarios?

```
def fun():
    global x
    x = 10
    return x
```

2. ¿Cuál es el problema con la siguiente función? Explore el resultado de las llamadas que se incluyen luego de la definición.

```
def agregar_usuario(usuario, listado=[]):
    listado.append(usuario)
    return listado

base1 = agregar_usuario("Adrián")
base2 = agregar_usuario("Daniela")
```

   Explore el contenido y la identidad de `base1` y `base2`. Si es necesario, utilice algún agente de IA para explorar en qué parte de la definición de la función se esconde una trampa.

1.2 Anónimas

1. ¿Por qué la siguiente definición devuelve un error?

```
lambda x: return x + 1
```

2. ¿Qué hacen los siguientes bloques de código?

```
(lambda x, y: x + y)(1, 5)
```

```
(lambda f, x: f(x + 10))(lambda y: y * 5, 2)
```

3. ¿Qué devuelve la siguiente llamada? ¿Por qué?

```
(lambda x: print(x))(1)
```

Ayuda: asigne el resultado a una variable y explore el valor de esa variable.

1.3 Variádicas

1. ¿Cuál es el valor de `args` en la siguiente llamada? ¿Por qué?

```
def fun(x, *args):
    return x + 10

fun(128)
```

¿Y el valor de `kwargs` debajo?

```
def fun(x, **kwargs):
    return x + 11

fun(128)
```

2. Los siguientes intentos por definir una función arrojan un error. Investigue por qué y reflexione sobre el sentido de los errores.

```
def fun(**kwargs, x, y):
    return x + y
```

```
def fun(**kwargs, *args):
    return sum(args)
```

3. Cualquier llamada a la siguiente función que solo pase argumentos posicionales va a resultar en un error. ¿Por qué? ¿Cómo habría que llamarla para que no resulte en un error?

```
def fun(*args, x, y):
    return sum(args), x + y
```

1.4 *Closures*

1. Ejecute el siguiente bloque de código e inspeccione el objeto que devuelven las sucesivas llamadas realizadas. Analice por qué ocurre este comportamiento y piense qué tipo de funcionalidades podrían aprovechar un mecanismo similar.

```
def fabrica():
    cosas = []
    def fun(elemento):
        cosas.append(elemento)
        return cosas
    return fun

f = fabrica()
f(128)
f(256)
f(1024)
```

Ayuda: Compare el ID de los objetos devueltos en cada llamada. ¿Podría observar el mismo comportamiento si en vez de usar una lista para `cosas` se usara una tupla?

2 Recursión

1. ¿Cuál de las siguientes funciones es recursiva?

```
def f(x):
    return x + 1

def g(x):
    return f(x)

def h(x):
    return h(x - 1)
```

2. ¿Qué son el caso base y el caso recursivo?
3. ¿Por qué toda recursión necesita un caso base?
4. ¿Qué pasa si una función recursiva nunca llega al caso base?
5. Reflexione sobre ventajas y desventajas de la recursión.

3 Funciones de orden superior

1. Explique por qué el primer `print` muestra `10` y el segundo falla:

```
from functools import partial

def suma(x, y):
```

```
    return x + y

suma2 = partial(suma, 10, 0)
print(suma2())
print(suma2(10))
```

2. ¿Cuál de las siguientes funciones es de orden superior?

```
def f(x):
    return x + 1

def g(fn, x):
    return fn(x)

def h():
    return f
```

3. ¿Cuál de las siguientes funciones de Python es de orden superior?

- `sum`
- `max`
- `min`
- `sorted`
- `print`
- `functools.partial`

4. ¿Qué hace la función `fun`? ¿Qué condicion debe cumplir `f` respecto de sus valores de entrada y salida?

```
def fun(f, x):
    return f(f(x))
```

5. ¿Para qué sirve la función definida debajo?

```
def fun(f, g):
    return lambda x: f(g(x))
```

6. ¿Qué tipo de función se necesita pasar a `filter`?
7. ¿Cuál es el resultado de la última línea de código? ¿Por qué?

```
filter_obj = filter(lambda x: x > 2, range(5))
list(filter_obj) # Primera conversión a lista
list(filter_obj) # Segunda conversión a lista
```

Ayuda: Para comprender lo que sucede en este ejercicio es necesario estar familiarizados con los **generadores**.

8. ¿Cuál implementación es más eficiente desde el punto de vista del consumo de memoria? ¿Por qué?

```
impares = list(filter(lambda x: x % 2, range(1_000)))
impares_al_cubo = list(map(lambda x: x ** 3, impares))
```

```
impares_al_cubo = list(map(lambda x: x ** 3, filter(lambda x: x % 2,
range(1_000))))
```

Ayuda: Nuevamente, es necesario estar familiarizados con los **generadores** para responder a esta pregunta.

9. ¿Cuál es el resultado de la reducción debajo? ¿Por qué?

```
from functools import reduce
reduce(lambda x, y: x + [y], range(3), [])
```

Ayuda: `reduce` ejecuta la función que se le pasa solamente 3 veces. Podría resultar de ayuda enumerar cada una de las llamadas manualmente.

4 Comprehensions

1. ¿Qué diferencia hay entre las siguientes dos líneas de código? Asuma que `f` es una función y `xs` es una secuencia.

```
map(f, xs)
[f(x) for x in xs]
```

2. ¿Cuál es el resultado de esta expresión?

```
[x for x in "hola" if x != "o"]
```

3. Escriba una *list comprehension* para obtener una lista con la primera letra de cada palabra.

```
palabras = ["hola", "mundo", "python"]
```

4. Escriba una *list comprehension* que reemplace los `None` por un `-1`.

```
datos = [1, None, 3, None, 5]
```

5. Convierta este bucle en una *list comprehension*.

```
resultado = []
for x in range(5):
    if x % 2 != 0:
        resultado.append(x)
```

6. ¿Es posible que una *list comprehension* devuelva una lista vacía? ¿Cuándo?
7. Construya una única *list comprehension* que realice lo mismo que el siguiente bloque:

```
def f(x):
    return 10 / x

def g(x):
    return x != 0

numeros = [1, 0, -1, 0, 2, 0, -2, 0, 3, 0, -3]
list(map(f, filter(g, numeros)))
```

5 Generadores

1. ¿Qué diferencia a una función regular de una función generadora?
2. ¿En qué momento se ejecuta el cuerpo de una función generadora? ¿Siempre se ejecuta todo?
3. ¿Cómo se puede obtener una lista a partir de un generador? ¿Cuántas veces se puede realizar esa operación sobre el mismo generador?
4. ¿Qué ventaja tiene un generador frente a una lista?
5. ¿Qué significa que un generador sea perezoso?
6. ¿Qué pasa si se llama a `next` sobre un generador más veces que la cantidad de `yield` en él?
7. ¿Qué se obtiene en el siguiente bloque de código?

```
def f():
    yield 1
    yield 2

gen = f()
print(list(gen))
```

   ¿Qué pasa si se corre `next(gen)` a continuación?

8. ¿Qué se imprime en pantalla?

```
print((x * 2 for x in [1, 2, 3]))
```

9. ¿Por qué es posible implementar un bucle infinito dentro de un generador?

10. ¿Cuántas veces es posible llamar a `next` sobre el generador `g`?

```
def fun():
    yield 1
    return 2
    yield 3

g = fun()
```